

Práctica Final

Reinforcement Learning - Deep Learning

Agente DQN

ASIGNATURA

OPT-ML

FORMATO

Jupyter Notebook

MODALIDAD

Individual / Grupal

Objetivo

Diseñar, implementar y entrenar un agente DQN (Deep Q-Network) que aprenda a resolver una tarea en un entorno de vuestra elección, aplicando los conocimientos adquiridos en Deep Learning y Reinforcement Learning durante el curso.

Objetivos de aprendizaje

- Comprender el ciclo agente-entorno y el framework MDP
- Implementar el algoritmo DQN con PyTorch
- Diseñar experimentos y analizar curvas de aprendizaje
- Aplicar técnicas de Deep Learning a problemas de control
- Documentar y comunicar resultados de forma clara

Descripción de la Tarea

El alumno/grupo debe entregar un **Jupyter Notebook (.ipynb)** completamente funcional donde:

1

Seleccionar o crear un entorno

Podéis elegir entre tres opciones: **crear vuestro propio juego con Pygame**, utilizar un juego/entorno encontrado en internet, o usar alguno de los **juegos de Atari** disponibles en gymnasium.farama.org. El entorno debe justificar el uso de DQN (estados de alta dimensionalidad o continuos).

2

Analizar el entorno

Documentar el espacio de estados, espacio de acciones, estructura de recompensas y condiciones de terminación. Incluir ejemplos de observaciones y episodios.

3

Implementar el agente DQN

Desarrollar un agente DQN con PyTorch incluyendo: red neuronal para aproximar $Q(s,a)$, Experience Replay Buffer y Target Network. Justificar la arquitectura y los hiperparámetros elegidos.

4

Entrenar y monitorizar

Ejecutar el entrenamiento registrando métricas: recompensa por episodio, pérdida, epsilon. Mostrar curvas de aprendizaje con media móvil.

5

Evaluar y analizar

Evaluar el agente entrenado (sin exploración) durante varios episodios. Analizar si el agente ha aprendido, comparar con baseline aleatorio, y discutir posibles mejoras.

Elección del Entorno

Tenéis libertad para elegir el entorno donde entrenar vuestro agente DQN. Las opciones son:

Opción 1: Crear vuestro propio juego con Pygame

Diseñar e implementar un juego desde cero usando Pygame. Esto os dará control total sobre el espacio de estados, acciones y recompensas. Ejemplos: Snake, juego de naves, laberinto, etc.

Opción 2: Usar un juego/entorno de internet

Buscar un entorno ya implementado en repositorios de GitHub o en otras fuentes.
Aseguraos de citar la fuente y adaptarlo si es necesario para que funcione con DQN.

Opción 3: Juegos de Atari (Gymnasium)

Usar los entornos clásicos de Atari disponibles en gymnasium.farama.org. Estos entornos son estándar en la investigación de RL y ofrecen un buen desafío.

```
pip install gymnasium[atari,accept-rom-license]
```

Componentes del Agente DQN

El agente DQN debe incluir los siguientes componentes fundamentales:

Componente	Descripción
Red Q (Q-Network)	Red neuronal en PyTorch que aproxima $Q(s, a)$. Recibe el estado y devuelve el Q-valor de cada acción.
Target Network	Copia de la red Q que se actualiza periódicamente. Proporciona estabilidad al calcular los targets.
Experience Replay	Buffer circular que almacena transiciones $(s, a, r, s', done)$. Se muestrea aleatoriamente para entrenar.
Política ϵ-greedy	Selección de acciones con exploración (random) y explotación (mejor Q). Epsilon debe decaer durante el entrenamiento.

Estructura del Entregable

El notebook debe seguir esta estructura mínima. **Podéis incluir archivos `.py` auxiliares** que encapsulen lógica (agente, entorno, utilidades) y se importen desde el notebook.

Sección	Contenido
1. Introducción	Nombre/s, fecha, entorno elegido, objetivo

Sección	Contenido
2. Análisis del Entorno	Espacios de estados/acciones, recompensas, episodio de ejemplo
3. Diseño del Agente DQN	Arquitectura de la red, hiperparámetros justificados
4. Entrenamiento	Código de entrenamiento con logging de métricas
5. Resultados	Curvas de aprendizaje, métricas finales
6. Evaluación	Test del agente entrenado, comparación con baseline
7. Conclusiones	Análisis, dificultades encontradas, posibles mejoras

Archivos a entregar

- `Apellido_Nombre_RL_Practica.ipynb` — Notebook principal (ejecutable)
- `*.py` — Archivos auxiliares opcionales (agente, entorno, utils...)
- `*.pth` — Modelo entrenado (opcional pero recomendado)

Rúbrica de Evaluación

Criterio	Peso	Descripción de niveles
Código Funcional	30%	Excelente (30) Código limpio, bien estructurado, ejecuta sin errores, DQN implementado correctamente
		Bueno (22) Funciona con errores menores, código organizado
		Suficiente (15) Funciona parcialmente, requiere correcciones
		Insuficiente (0-10) No ejecuta o errores graves en DQN
Curvas de Aprendizaje	20%	Excelente (20) Gráficas claras con media móvil, múltiples métricas (reward, loss, epsilon), análisis de tendencias
		Bueno (15) Gráfica de recompensa vs episodio correcta
		Suficiente (10) Gráfica básica sin procesamiento
		Insuficiente (0-5) Sin visualización o incorrecta

Criterio	Peso	Descripción de niveles
Evaluación del Agente	20%	Excelente (20) Evaluación rigurosa (N episodios), comparación con baseline aleatorio, métricas estadísticas (media, std) Bueno (15) Evaluación correcta con resultados claros Suficiente (10) Evaluación básica sin comparativa Insuficiente (0-5) Sin evaluación o incorrecta
Documentación	20%	Excelente (20) Explicaciones claras, hiperparámetros justificados, análisis profundo de resultados y decisiones de diseño Bueno (15) Buena documentación, justificaciones presentes Suficiente (10) Documentación mínima Insuficiente (0-5) Sin documentación o incomprensible
Creatividad y Calidad	10%	Excelente (10) Entorno original/interesante, análisis de hiperparámetros, experimentación adicional, código bien modularizado Bueno (7) Algún elemento extra o análisis adicional Suficiente (4) Trabajo estándar sin extras Insuficiente (0-2) Trabajo mínimo
100 puntos Total máximo (sin bonificaciones)		

Consejos y Recursos

Tips para el éxito

- **Empieza simple:** Probad primero con un entorno sencillo para verificar que DQN funciona
- **Usa semillas:** `env.reset(seed=42)` y `torch.manual_seed(42)` para reproducibilidad
- **Loggea métricas:** Guardad recompensas, pérdidas y epsilon por episodio
- **Entrena sin render:** Desactivad la visualización durante el entrenamiento (mucho más rápido)
- **Paciencia:** El RL es lento, los agentes pueden tardar miles de episodios en aprender

- **Experimenta con hiperparámetros:** Probad diferentes learning rates (0.001, 0.0003, 0.0001)
- **Buffer grande:** Usad un replay buffer de al menos 10,000 experiencias
- **Decaimiento de epsilon:** Aseguraos de que epsilon llegue a valores bajos (~0.01) al final

Recursos útiles

- **Documentación Gymnasium:** gymnasium.farama.org
- **Documentación PyTorch:** pytorch.org/docs
- **Hugging Face Deep RL Course:** huggingface.co/learn/deep-rl-course
- **Documentación Pygame:** pygame.org/docs
- **Notebooks del curso:** `01_teoría/RL_02_dqn.ipynb`
- **Ejemplos del curso:** `02_fundamentos/ejemplos/ejemplo_cartpole_dqn.py`
- **Paper original DQN:** [Human-level control through deep reinforcement learning \(2015\)](https://arxiv.org/abs/1312.5680)

Bonificaciones

Puntos extra (hasta +20%)

Bonus	Puntos	Descripción
Entorno propio con Pygame	+10%	Crear un juego/entorno original desde cero con Pygame
Mejoras a DQN	+5%	Implementar Double DQN, Dueling DQN o Prioritized Experience Replay
Análisis de hiperparámetros	+5%	Comparativa sistemática variando learning rate, arquitectura, buffer size, etc.
Visualización del agente	+5%	Vídeo o GIF del agente entrenado jugando, o visualización de Q-valores

Las bonificaciones son acumulables hasta un máximo de +20% sobre la nota base.

Formato de Entrega

Archivo principal	<code>Apellido_Nombre_RL_Practica.ipynb</code> (o <code>Grupo_X_RL_Practica.ipynb</code>)
Archivos auxiliares	Archivos <code>.py</code> con código modularizado (agente, entorno, utils...)
Modelo	Archivo <code>.pth</code> con los pesos del modelo entrenado (opcional)
Requisito	El notebook debe ejecutarse de principio a fin sin errores (<i>Kernel</i> → <i>Restart & Run All</i>)

OPT-ML | Deep Learning & Reinforcement Learning

Materiales en: [RL_02_dqn.ipynb](#) | [Ejemplos](#)